

Grid BFS + Path Recovery

```
// https://cses.fi/problemset/result/14125885/
// CSES - Graphs - Labyrinth

#include <bits/stdc++.h>

using namespace std;

const char WALL = '#';
const char START = 'A';
const char END = 'B';
const int MAX = 1005;

using pi = pair<int, int>;

int n, m;

pi start, End;

char mtx[MAX][MAX];
bool visited[MAX][MAX];
pi parent[MAX][MAX];

int counter = 0;

string traceback() {
    string res;
    auto prev = End;
    while (prev != start) {
        counter++;
        const auto [px, py] = prev;
        const auto [x, y] = parent[px][py];
        if (x + 1 == px && y == py)
            res.push_back('D');
        if (x - 1 == px && y == py)
            res.push_back('U');
        if (x == px && y + 1 == py)
            res.push_back('R');
        if (x == px && y - 1 == py)
            res.push_back('L');
        prev = {x, y};
    }
    return res;
}

bool valid(int x, int y) {
    return !(x < 0 || y < 0 || x == n || y == m || mtx[x][y] == WALL);
}

bool bfs() {
    queue<pi> q;
    q.push(start);
    visited[start.first][start.second] = true;
    while (!q.empty()) {
        const auto curr = q.front();
        q.pop();
        const auto [x, y] = curr;
        if (curr == End)
            return true;
        pi options[4] = {{x + 1, y}, {x - 1, y}, {x, y + 1}, {x, y - 1}};
        for (const auto &[nx, ny] : options) {
            if (valid(nx, ny) && !visited[nx][ny]) {
                visited[nx][ny] = true;
                parent[nx][ny] = {x, y};
                q.push({nx, ny});
            }
        }
    }
    return false;
}

int main(int argc, char *argv[]) {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
    cin >> n >> m;
    for (size_t i = 0; i < n; i++) {
        for (size_t j = 0; j < m; j++) {
            char aux;
            cin >> aux;
```

```
            mtx[i][j] = aux;
            if (aux == START)
                start = {i, j};
            if (aux == END)
                End = {i, j};
        }
    }

    bool found = bfs();
    if (!found) {
        cout << "NO" << endl;
        return 0;
    }
    cout << "YES" << endl;
    string res = traceback();
    cout << counter << endl;
    reverse(res.begin(), res.end());
    cout << res << endl;

    return 0;
}
```

Flood Fill / Components

```
#include <bits/stdc++.h>
using namespace std;
```

```
using pii = pair<int, int>;
```

```
const int dx[] = {-1, 1, 0, 0};
const int dy[] = {0, 0, -1, 1};
```

```
int floodFill(vector<string> &g, int sx, int sy, char from = '.',
              char to = '#') {
    int n = g.size(), m = g[0].size();
    if (g[sx][sy] != from)
        return 0;
```

```
    queue<pii> q;
    q.push({sx, sy});
    g[sx][sy] = to;
```

```
    int cnt = 0;
```

```
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        cnt++;
```

```
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
            int ny = y + dy[d];
```

```
            if (nx < 0 || nx >= n || ny < 0 || ny >= m)
                continue;
            if (g[nx][ny] != from)
                continue;
```

```
            g[nx][ny] = to;
            q.push({nx, ny});
        }
    }
```

```
    return cnt;
}
```

```
void solve() {
    int n, m;
    cin >> n >> m;
```

```
    vector<string> g(n);
    for (auto &row : g)
        cin >> row;
```

```
    int components = 0;
```

```
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (g[i][j] == '.') {
                floodFill(g, i, j);
                components++;
```

```
            }

            cout << components << '\n';
        }
    }
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    solve();
}
```

Kosaraju SCC

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
void dfs1(int v, const vector<vector<int>> &g, vector<bool> &vis,
          vector<int> &order) {
    vis[v] = true;
    for (int u : g[v])
        if (!vis[u])
            dfs1(u, g, vis, order);
    order.push_back(v);
}
```

```
void dfs2(int v, const vector<vector<int>> &rg, vector<bool> &vis) {
    vis[v] = true;
    for (int u : rg[v])
        if (!vis[u])
            dfs2(u, rg, vis);
}
```

```
int kosaraju(const vector<vector<int>> &g, const vector<vector<int>> &rg) {
    int n = (int)g.size() - 1;
```

```
    vector<bool> vis(n + 1);
    vector<int> order;
    order.reserve(n);
```

```
    for (int v = 1; v <= n; v++)
        if (!vis[v])
            dfs1(v, g, vis, order);
```

```
    fill(vis.begin(), vis.end(), false);
```

```
    int scc = 0;
    reverse(order.begin(), order.end());
```

```
    for (int v : order) {
        if (vis[v])
            continue;
        dfs2(v, rg, vis);
        scc++;
    }
```

```
    return scc;
}
```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```
    int n, m;
```

```
    while (cin >> n >> m && (n || m)) {
        vector<vector<int>> g(n + 1), rg(n + 1);
```

```
        for (int i = 0; i < m; i++) {
            int x, y, z;
            cin >> x >> y >> z;
```

```
            g[x].push_back(y);
            rg[y].push_back(x);
```

```
            if (z == 2) {
                g[y].push_back(x);
                rg[x].push_back(y);
            }
        }
    }
```

```
    cout << (kosaraju(g, rg) == 1) << '\n';
}
}
```

Subset Sum: Exactly K

// Subset sum com uma determinada quantidade de elementos

```
#include <bits/stdc++.h>
using namespace std;
```

```
using ll = long long;
```

```
int main() {
    int n;
    cin >> n;

    vector<int> a(n);
    for (int &x : a)
        cin >> x;

    const int K = 4;

    vector<unordered_map<int, ll>> dp(K + 1);
    dp[0][0] = 1;
```

```
    for (int x : a)
        for (int k = K - 1; k >= 0; k--)
            for (auto [sum, cnt] : dp[k])
                dp[k + 1][sum + x] += cnt;
```

```
    int target;
    cin >> target;
```

```
    cout << dp[K][target] << '\n';
}
```

Interactive Binary Search

```
// regra numero 1:
//
// COLOCA BOUND EXATO. SE O T VAI DE O ATE 100, L = 0 E R = 100
//
//
// regra numero 2:
//
// VE SE A STRING TA CERTA ANTES DE MANDAR, PQ VCS VAO DIMINUIR ELA PRA TESTAR E
// SE FERRAR
//
//
// regra numero 3:
//
// 'WRONG ANSWER' EH A MESMA COISA QUE 'TIME LIMIT EXCEED' NESSES PROBLEMAS
```

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
int main() {
    ll l = 1, r = 1000000000;
    // ll l = 1, r = 100;
    string s;
```

```
    while (l < r) {
        ll mid = (l + r) / 2;
        cout << "?" << mid << endl;
        cin >> s;
```

```
        if (s == "QUENTE")
            l = mid + 1;
        else
            r = mid;
    }
    cout << "!" << r << endl;
}
```

Equal-Value Subarrays

*// numero de substrings num intervalo de tamanho n = (n * (n+1))/2*
// problema:
// https://codeforces.com/contest/84/problem/B (nome: Magical Array)

// solucao:

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
```

```
int main() {
    ll n;
    cin >> n;
    vector<ll> a(n);
    for (auto &i : a)
        cin >> i;
    // formula = (x * (x+1)) / 2
    ll asw = 0;
    ll aux = 1e12;
    ll seq = 0;
    for (int i = 0; i < n; i) {
        if (a[i] != aux) {
            aux = a[i];
            seq = 0;
        }
        while (i < n && a[i] == aux) {
            seq++;
            i++;
        }
        asw += (seq * (seq + 1)) / 2;
    }
    cout << asw << endl;
}
```

Lazy SegTree: Add / Max

```
#include <bits/stdc++.h>
#define ll long long
```

```
using namespace std;
#define MAX 100010
```

```
ll v[MAX];
ll seg[4 * MAX];
ll lazy[4 * MAX]; // armazena multiplicacoes pendentes
```

```
ll build(int p, int l, int r) {
    lazy[p] = 0; // neutro da soma
    if (l == r)
        return seg[p] = v[l];
    int m = (l + r) / 2;
    return seg[p] = max(build(2 * p, l, m), build(2 * p + 1, m + 1, r));
}
```

```
// Propaga o valor pendente do no atual para seus filhos
void push(int p, int l, int r) {
    if (lazy[p] != 0) {
```

```
        seg[2 * p] += lazy[p];
        seg[2 * p + 1] += lazy[p];
```

```
        lazy[2 * p] += lazy[p];
        lazy[2 * p + 1] += lazy[p];
```

```
        lazy[p] = 0;
    }
}
```

// Atualiza o intervalo [a, b] somando x a todos os elementos

```
void update(int a, int b, ll x, int p, int l, int r) {
    push(p, l, r);
    if (b < l || r < a)
        return;
    if (a <= l && r <= b) {
        seg[p] += x;
        lazy[p] += x;
        return;
    }
```

```
    push(p, l, r);
```

```
    int m = (l + r) / 2;
    update(a, b, x, 2 * p, l, m);
    update(a, b, x, 2 * p + 1, m + 1, r);
    seg[p] = max(seg[2 * p], seg[2 * p + 1]);
}
```

```
}
```

// Consulta a soma no intervalo [a, b]

```
ll query(int a, int b, int p, int l, int r) {
    push(p, l, r);
    if (b < l || r < a)
        return 0; // neutro da soma
    if (a <= l && r <= b)
        return seg[p];
    int m = (l + r) / 2;
    return max(query(a, b, 2 * p, l, m), query(a, b, 2 * p + 1, m + 1, r));
}
```

```
int main(int argc, char *argv[]) {
    cin.tie(0)->sync_with_stdio(0);
```

```
    int tt = 1;
    // cin >> tt;
```

```
    while (tt--) {
        int n, q;
        cin >> n >> q;
```

```
        memset(v, 0, sizeof(v));
        memset(seg, 0, sizeof(seg));
        memset(lazy, 0, sizeof(lazy));
```

```
        build(1, 0, n - 1);
```

```
        while (q--) {
            int o;
            cin >> o;
            if (o == 1) {
                int p, q;
                ll val;
                cin >> p >> q >> val;
                p--;
                q--;
                update(p, q, val, 1, 0, n - 1);
            } else {
                int p, r;
                cin >> p >> r;
                p--;
                r--;
                ll res = query(p, r, 1, 0, n - 1);
                cout << res << endl;
            }
        }
    }
```

```
    return 0;
}
```

Configurable Lazy SegTree

```
#include <bits/stdc++.h>
using namespace std;
```

```
using ll = long long;
```

```
const ll MOD = 1e9 + 9;
```

```
// CURRENT CONFIGURATION:
// Query: MAX
// Update: RANGE ADD
//
// Valid ADD queries: SUM, SUM MOD, MAX, MIN
// Valid SET queries: SUM, SUM MOD, MAX, MIN, GCD, XOR, MULT
```

```
struct SegTree {
    int n;
    vector<ll> tree, lazy;
```

```
    // Needed for RANGE SET lazy:
    // vector<char> hasLazy;
```

```
    SegTree(const vector<ll> &a) {
        n = a.size();
        tree.resize(4 * n);
        lazy.assign(4 * n, 0);
    }
```

```

// RANGE SET:
// hasLazy.assign(4 * n, false);

if (n)
    build(1, 0, n - 1, a);
}

// Identity value for queries outside the range.
ll neutral() {
    // SUM:
    // return 0;

    // MAX:
    return LLONG_MIN;

    // MIN:
    // return LLONG_MAX;

    // GCD:
    // return 0;

    // XOR:
    // return 0;

    // MULT:
    // return 1;
}

// Merge two children.
ll mergeVal(ll a, ll b) {
    // SUM:
    // return a + b;

    // SUM MOD:
    // return (a + b) % MOD;

    // MAX:
    return max(a, b);

    // MIN:
    // return min(a, b);

    // GCD:
    // return gcd(a, b);

    // XOR:
    // return a ^ b;

    // MULT:
    // return a * b % MOD;
}

// Modular exponentiation -- used by MULT.
ll power(ll a, int e) {
    a %= MOD;
    if (a < 0)
        a += MOD;

    ll res = 1;

    while (e) {
        if (e & 1)
            res = res * a % MOD;

        a = a * a % MOD;
        e >>= 1;
    }

    return res;
}

continued
void build(int p, int l, int r, const vector<ll> &a) {
    if (l == r) {
        // SUM / MAX / MIN / XOR:
        tree[p] = a[l];

        // SUM MOD:
        // tree[p] = (a[l] % MOD + MOD) % MOD;

        // GCD:

```

```

        // tree[p] = abs(a[l]);

        // MULT:
        // tree[p] = (a[l] % MOD + MOD) % MOD;
    }

    return;
}

int m = (l + r) / 2;

build(p * 2, l, m, a);
build(p * 2 + 1, m + 1, r, a);

tree[p] = mergeVal(tree[p * 2], tree[p * 2 + 1]);
}

// Apply lazy update to every element in [l, r].
void apply(int p, int l, int r, ll x) {
    int len = r - l + 1;

    //////////////////////////////////////
    // RANGE ADD -- COMMENT THIS WHOLE BLOCK WHEN USING RANGE SET
    //////////////////////////////////////

    // SUM + ADD:
    // tree[p] += x * len;

    // SUM MOD + ADD:
    // tree[p] = (tree[p] + (x % MOD + MOD) % MOD * len) % MOD;

    // MAX + ADD:
    tree[p] += x;

    // MIN + ADD:
    // tree[p] += x;

    // Lazy composition for ADD:
    lazy[p] += x;

    //////////////////////////////////////
    // RANGE SET -- COMMENT THIS WHOLE BLOCK WHEN USING RANGE ADD
    //////////////////////////////////////

    // SUM + SET:
    // tree[p] = x * len;

    // SUM MOD + SET:
    // tree[p] = (x % MOD + MOD) % MOD * len % MOD;

    // MAX + SET:
    // tree[p] = x;

    // MIN + SET:
    // tree[p] = x;

    // GCD + SET:
    // tree[p] = abs(x);

    // XOR + SET:
    // tree[p] = (len & 1) ? x : 0;

    // MULT + SET:
    // tree[p] = power(x, len);

    // Lazy composition for SET:
    // lazy[p] = x;
    // hasLazy[p] = true;
}

continued
void push(int p, int l, int r) {
    if (l == r)
        return;

    //////////////////////////////////////
    // RANGE ADD
    //////////////////////////////////////

    if (lazy[p] == 0)
        return;

```

```

    int m = (l + r) / 2;

    apply(p * 2, l, m, lazy[p]);
    apply(p * 2 + 1, m + 1, r, lazy[p]);

    lazy[p] = 0;

    //////////////////////////////////////
    // RANGE SET
    //////////////////////////////////////

    /*
    if (!hasLazy[p])
        return;

    int m = (l + r) / 2;

    apply(p * 2, l, m, lazy[p]);
    apply(p * 2 + 1, m + 1, r, lazy[p]);

    hasLazy[p] = false;
    */
}

// Apply update x to every a[i] in [ql, qr].
void update(int p, int l, int r, int ql, int qr, ll x) {
    if (qr < l || r < ql)
        return;

    if (ql <= l && r <= qr) {
        apply(p, l, r, x);
        return;
    }

    push(p, l, r);

    int m = (l + r) / 2;

    update(p * 2, l, m, ql, qr, x);
    update(p * 2 + 1, m + 1, r, ql, qr, x);

    tree[p] = mergeVal(tree[p * 2], tree[p * 2 + 1]);
}

ll query(int p, int l, int r, int ql, int qr) {
    if (qr < l || r < ql)
        return neutral();

    if (ql <= l && r <= qr)
        return tree[p];

    push(p, l, r);

    int m = (l + r) / 2;

    return mergeVal(query(p * 2, l, m, ql, qr),
                    query(p * 2 + 1, m + 1, r, ql, qr));
}

void update(int l, int r, ll x) {
    if (n)
        update(1, 0, n - 1, l, r, x);
}

ll query(int l, int r) {
    if (!n)
        return neutral();

    return query(1, 0, n - 1, l, r);
}

};

int main() {
    vector<ll> a = {1, 2, 3, 4, 5};
    SegTree st(a);

    cout << st.query(1, 3) << '\n'; // 4

    // a[1..3] += 10
    st.update(1, 3, 10);
}

```

```
    cout << st.query(1, 3) << '\n'; // 14
}
```

Knapsack + Recovery

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
using ll = long long;
```

```
struct Result {
    ll best;
    vector<int> ans;
};
```

```
////////////////////////////////////
// -- KNAPSACK 0/1 + RECUPERACAO -- //
////////////////////////////////////
```

```
Result knapsack01(int n, int W, const vector<int> &w, const vector<ll> &val) {
    // dp[p] = maior valor usando peso <= p
```

```
    vector<ll> dp(W + 1);
    vector<vector<char>> take(n + 1, vector<char>(W + 1));
```

```
    for (int i = 1; i <= n; i++)
        for (int p = W; p >= w[i]; p--)
            if (dp[p - w[i]] + val[i] > dp[p]) {
                dp[p] = dp[p - w[i]] + val[i];
                take[i][p] = 1;
            }
```

```
    ll best = dp[W]; // resposta
```

```
    // Recupera itens
    vector<int> ans;
    int p = W;
```

```
    for (int i = n; i >= 1; i--)
        if (take[i][p]) {
            ans.push_back(i);
            p -= w[i];
        }
```

```
    reverse(ans.begin(), ans.end());
```

```
    return {best, ans};
```

```
}
```

```
continued
////////////////////////////////////
// -- KNAPSACK COMPLETO (PODE REPETIR ITEM) + RECUPERACAO -- //
////////////////////////////////////
```

```
Result knapsackComplete(int n, int W, const vector<int> &w,
                        const vector<ll> &val) {
    // dp[p] = maior valor usando peso <= p
```

```
    vector<ll> dp(W + 1);
    vector<vector<char>> take(n + 1, vector<char>(W + 1));
```

```
    for (int i = 1; i <= n; i++)
        for (int p = w[i]; p <= W; p++)
            if (dp[p - w[i]] + val[i] > dp[p]) {
                dp[p] = dp[p - w[i]] + val[i];
                take[i][p] = 1;
            }
```

```
    ll best = dp[W]; // resposta
```

```
    // Recupera itens
    vector<int> ans;
    int i = n, p = W;
```

```
    while (i >= 1) {
        if (take[i][p]) {
            ans.push_back(i);
            p -= w[i]; // continua no mesmo i
        } else {
```

```
            i--; // item i nao foi usado aqui
        }
    }
```

```
    reverse(ans.begin(), ans.end());
```

```
    return {best, ans};
}
```

```
continued
////////////////////////////////////
// -- KNAPSACK COMPLETO (PODE REPETIR ITEM) VALOR + RECUPERACAO -- //
////////////////////////////////////
```

```
Result knapsackCompleteValue(int n, int W, const vector<int> &w,
                             const vector<ll> &val) {
    // dp[v] = menor peso para obter valor exatamente v
```

```
    // Limite superior seguro para a resposta
    int soma_valores = 0;
```

```
    for (int i = 1; i <= n; i++)
        soma_valores = max(soma_valores, (int)((__int128)W * val[i] / w[i]));
```

```
    const ll INF = LLONG_MAX / 4;
```

```
    vector<ll> dp(soma_valores + 1, INF);
    vector<int> par(soma_valores + 1, -1);
```

```
    dp[0] = 0;
```

```
    for (int i = 1; i <= n; i++)
        for (int v = val[i]; v <= soma_valores; v++)
            if (dp[v - val[i]] != INF && dp[v - val[i]] + w[i] < dp[v]) {
                dp[v] = dp[v - val[i]] + w[i];
                par[v] = i;
            }
```

```
    int best = soma_valores;
    while (dp[best] > W)
        best--;
```

```
    // Recupera itens
    vector<int> ans;
    int v = best;
```

```
    while (v > 0) {
        int i = par[v];
        ans.push_back(i);
        v -= val[i];
    }
```

```
    reverse(ans.begin(), ans.end());
```

```
    return {best, ans};
```

```
}
```

```
continued
////////////////////////////////////
// -- KNAPSACK 0/1 VALOR + RECUPERACAO -- //
////////////////////////////////////
```

```
Result knapsack01Value(int n, int W, const vector<int> &w,
                       const vector<ll> &val) {
    // dp[v] = menor peso para obter valor exatamente v
```

```
    int soma_valores = accumulate(val.begin() + 1, val.end(), 0LL);
```

```
    const ll INF = LLONG_MAX / 4;
```

```
    vector<ll> dp(soma_valores + 1, INF);
    vector<vector<char>> take(n + 1, vector<char>(soma_valores + 1));
```

```
    dp[0] = 0;
```

```
    for (int i = 1; i <= n; i++)
        for (int v = soma_valores; v >= val[i]; v--)
            if (dp[v - val[i]] != INF && dp[v - val[i]] + w[i] < dp[v]) {
                dp[v] = dp[v - val[i]] + w[i];
                take[i][v] = 1;
            }
```

```
    int best = soma_valores;
    while (dp[best] > W)
        best--;
```

```
    // Recupera itens
    vector<int> ans;
    int v = best;
```

```
    for (int i = n; i >= 1; i--)
        if (take[i][v]) {
            ans.push_back(i);
            v -= val[i];
        }
```

```
    reverse(ans.begin(), ans.end());
```

```
    return {best, ans};
```

```
}
```

```
continued
////////////////////////////////////
// -- DRIVER BASICO -- //
////////////////////////////////////
```

```
void printResult(const string &name, const Result &r) {
    cout << name << '\n';
    cout << "best = " << r.best << '\n';
```

```
    cout << "items:";
    for (int i : r.ans)
        cout << ' ' << i;
```

```
    cout << "\n\n";
```

```
}
```

```
int main() {
    int n = 3;
    int W = 7;
```

```
    // 1-indexado: posicao 0 nao e usada
    vector<int> w = {0, 3, 4, 2};
    vector<ll> val = {0, 4, 5, 3};
```

```
    printResult("0/1 por peso", knapsack01(n, W, w, val));
```

```
    printResult("Completo por peso", knapsackComplete(n, W, w, val));
```

```
    printResult("Completo por valor", knapsackCompleteValue(n, W, w, val));
```

```
    printResult("0/1 por valor", knapsack01Value(n, W, w, val));
```

```
}
```